

File System → (1) How data is organized.

{ (2) Actual Scheduling of Requests
Should also ensure fast access.

→ Block I/O Scheduler

↳ Schedule requests in such a way that the throughput of I/O is maximized.

↳ Reduce starvation.

FIFO → Low throughput because the R/W head might have to move a lot.
Throughput is the number of requests handled

per unit time

Linus Elevator: → Adjacent requests should be merged into a single request.

(1) Within the queue, are there any requests with which the current request could be handled together?
⇒ A ~~small~~ successful merge would require very small amount of movement of read/write head.

There could be two types of merges: →

1) Merging with a request to the back of the current one → Back merge (common)

2) Merging with a request to the front of the current one → Front merge (rare).

3) Sorting → Sort and check where in the middle it can be inserted.

~~54~~

55

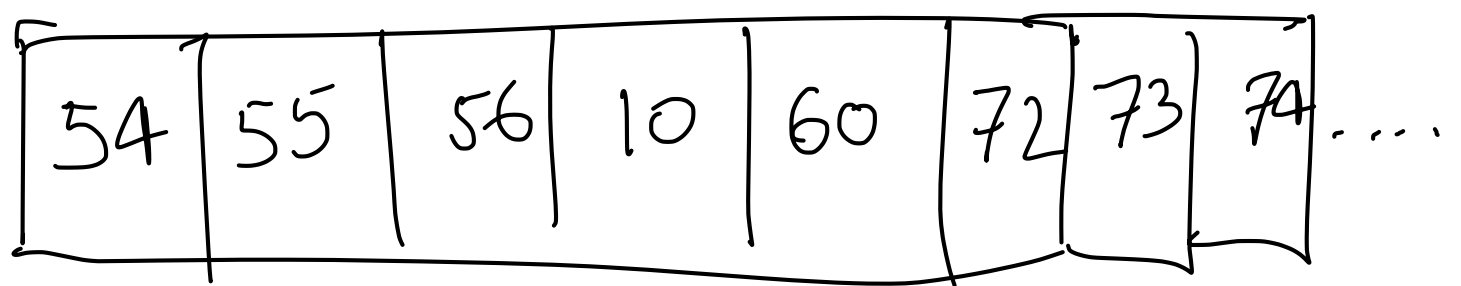
56

Only if sorting also shows that there is no location in the middle, the request is inserted into the tail of the queue.

Linux 2.4 → Linus Elevator

What are the drawbacks?

→ Infinite delay before reaching the tail of the queue is possible.



Was there anything possible to solve this problem?

→ Assign a deadline to each request.
Follow the Elevator Approach until some request

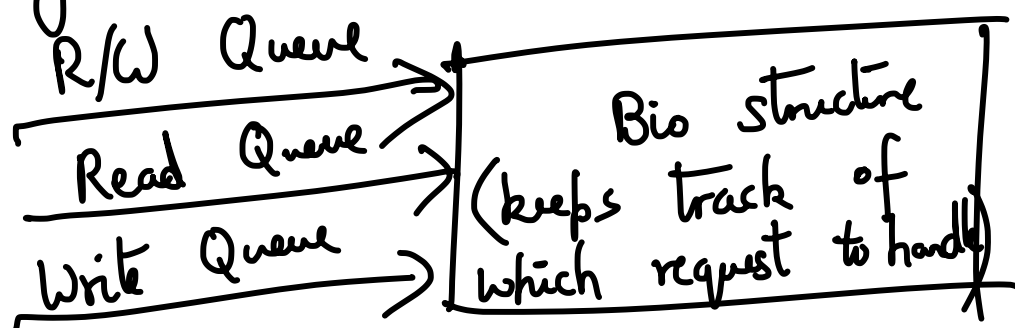
is close to its deadline. Then, move the disk read/write head to handle that deadline.

Deadline I/O Scheduler : $\rightarrow$ No request to the block device is ever starved. But handling one deadline can take so much time that it triggers the deadlines of other requests. This can force the r/w head to keep handling deadlines which lowers the throughput. This is called "seek storm". $\rightarrow$ Priority inversion is common (why?)

Is a better strategy possible?

Prioritization of read requests

Anticipatory I/O Scheduler $\rightarrow$



Write queues would be assigned higher deadlines than the read queues.

When any queue reaches close to the deadline, then its request is handled. Otherwise, the standard elevator technique is used for the R/W queue.

Linux has the virtual file system. But how is all this information about disks and other system info accessible to the user?

`/dev/sda` } → Each individual storage device
`/dev/sdb` } has a file associated with it.

Seek requests are allowed on only some files, which correspond to block devices.

Other information is also exposed by the virtual file system.

`/proc` . . .
It is a pseudo file system which exposes data about the system such as running processes, CPU usage, memory usage and even hardware.

`/proc/cpuinfo`

`/proc/meminfo`

Partitioning and formatting the disks is essential to create the superblocks and inodes.

Mounting the disk is about giving access to the disk from a particular directory.

/etc/fstab → Keeps track of all the disks

/etc/mntab → Keeps track of the mount location of the disks.

File systems also have permissions.

read, write, execute } Directories also have the same permissions.
owner, group, other



A convenient mechanism of providing access,

chown system call can be used to change the owner of a file.

chgrp system call to remove or add users to a group.

chmod system call to change permissions of a file.

Every user has a home directory. What does that

mean?

⇒ You create a directory owned by that user.
The user has permission to change or add content to all files in that directory.

All this information about files (owner, permissions) are stored in inodes.

The current directory → .

Previous
(Parent) → ..